← DevLab 使用說明

實例：中鴻週報戰情室容器化

把一個在 Windows 上直接執行的 Python 程式，轉成 Docker 容器並部署上線的完整紀錄（2026-10-14）。

1. 原本的樣子 2. 盤點 3. 設定檔 4. 建置 5. 測試 6. 憑證問題 7. 切換上線 8. 上線後 檢查清單

成果摘要

	轉換前	轉換後
網址	`http://10.125.2.14:5004`	`http://chs-warroom.10.125.2.14.sslip.io`
執行方式	Windows 工作排程器，開機時以 SYSTEM 身分執行 Python	Docker 容器，非 root 使用者
環境	主機上的 Python 3.11 + 虛擬環境，瀏覽器路徑寫死在使用者資料夾	映像檔內含 Python 3.12、Chromium、中文字型，可在任何 Docker 主機重建
資料	`D:\StockWarRoomV2\data\warroom.db`	Docker volume `chs-warroom_data`，每天自動備份
對外開放	5004 埠，防火牆允許**任何來源**；Host 檢查含 `*`（全部放行）	只經反向代理的 80 埠，限定網段；Host 只接受自己的網址
原始碼	**完全沒改**：Docker 相關檔案都放在另一個資料夾	
停機時間	約 1 分鐘	

1 原本的樣子

戰情室是 Flask 網頁程式，內建排程定時抓證交所資料，並用 Playwright（Chromium）擷取 MoneyDJ 的圖表。原專案資料夾：

```
D:\StockWarRoomV2\
├── run.py                    啟動程式（waitress，預設 5004 埠）
├── requirements-lock.txt     套件版本（Flask、APScheduler、playwright 1.59.0…）
├── warroom\                  主程式：app.py、jobs.py（排程）、pipeline.py、charts.py（截圖）
├── templates\  static\       網頁
├── data\warroom.db           SQLite 資料庫（WAL 模式，約 6 MB）
├── .venv\                    Python 虛擬環境（162 MB）
└── start_server.bat  setup_server.bat
```

複製

它不是用 `start_server.bat` 啟動的，而是**工作排程器**裡的 `\StockWarRoomV2`：開機觸發、以 SYSTEM 身分執行。這是用系統管理員權限查程序來源才發現的：

```
proc 4300 owner=NT AUTHORITY\SYSTEM parent=3816
  parent 3816 svchost.exe cmd=C:\WINDOWS\system32\svchost.exe -k netsvcs -p -s Schedule
task \StockWarRoomV2 state=Running user=SYSTEM trigger=MSFT_TaskBootTrigger
     exec=D:\StockWarRoomV2\.venv\Scripts\python.exe args=-X utf8 D:\StockWarRoomV2\run.py --host 0.
```

複製

為什麼要查這個？如果只在畫面上關掉程式，下次開機工作排程器又會把它啟動，變成原生版和容器版**同時跑排程**，兩邊重複抓資料、各自寫入不同的資料庫。

2 盤點：容器化要注意什麼

讀過 `run.py`、`app.py`、`jobs.py`、`charts.py` 後，整理出影響容器化的地方：

發現	風險	做法
Playwright 用 Chromium 截取中文網頁	容器沒有瀏覽器和中文字型，截圖會變方框	用 Playwright 官方映像檔（版本與 `requirements-lock.txt` 一致），再加裝 Noto CJK 字型
`run.py` 寫死 `PLAYWRIGHT_BROWSERS_PATH=C:\Users\CHSAI\…`	Linux 沒有這個路徑	程式用 `setdefault`，所以在映像檔設好環境變數就會優先使用，**不用改程式**

SQLite 開啟 WAL 模式	放在 Windows 資料夾掛載（bind mount）上可能鎖定或損壞	改放 Docker volume
程式內建 APScheduler 排程	同時跑兩份會重複執行工作	切換時先停用原生版；測試時把副本的排程全部停用
`ALLOWED_HOSTS` 含 `*`	Host 檢查形同虛設	容器版只允許自己的網址
資料庫從環境變數 `WARROOM_DB` 讀取路徑	—	設成 `/data/warroom.db`，指向 volume

3 寫設定檔

所有 Docker 相關檔案都放在 `D:\lab\apps\chs-warroom\`，**原專案資料夾一個檔案都不動**，原作者照常開發，不受影響。

檔案	用途
`Dockerfile`	映像檔內容
`Dockerfile.dockerignore`	只把需要的檔案送進建置（排除 `.venv`、資料庫、測試截圖）
`compose.yml`、`.env`	執行設定：網址、volume、資源上限
`build.ps1`	建置映像檔（原因見第 4 步）
`certs\chs-rootca.crt`	公司根憑證（原因見第 6 步）
`cutover.ps1`、`backup.ps1`	切換與備份

Dockerfile（最終版）

```
FROM mcr.microsoft.com/playwright/python:v1.59.0-noble

# 繁體中文字型：MoneyDJ 圖資截圖需要；libnss3-tools 提供 certutil
RUN apt-get update \
 && apt-get install -y --no-install-recommends fonts-noto-cjk python3-venv libnss3-tools \
```

```
 && rm -rf /var/lib/apt/lists/*

# 公司 HTTPS 檢查根憑證（見第 6 步）
COPY --from=certs chs-rootca.crt /usr/local/share/ca-certificates/chs-rootca.crt
RUN update-ca-certificates \
 && mkdir -p /home/pwuser/.pki/nssdb \
 && certutil -d sql:/home/pwuser/.pki/nssdb -N --empty-password \
 && certutil -d sql:/home/pwuser/.pki/nssdb -A -t "C,," -n CHS-RootCA -i /usr/local/share/ca-certifi
 && chown -R pwuser:pwuser /home/pwuser/.pki

ENV VIRTUAL_ENV=/opt/venv \
    PATH=/opt/venv/bin:$PATH \
    PYTHONUTF8=1 \
    PYTHONDONTWRITEBYTECODE=1 \
    PYTHONUNBUFFERED=1 \
    TZ=Asia/Taipei \
    PLAYWRIGHT_BROWSERS_PATH=/ms-playwright \
    REQUESTS_CA_BUNDLE=/etc/ssl/certs/ca-certificates.crt \
    SSL_CERT_FILE=/etc/ssl/certs/ca-certificates.crt \
    WARROOM_DB=/data/warroom.db

WORKDIR /app
COPY requirements-lock.txt .
RUN python3 -m venv $VIRTUAL_ENV && pip install --no-cache-dir -r requirements-lock.txt

COPY run.py ./
COPY warroom ./warroom
COPY templates ./templates
COPY static ./static

RUN mkdir -p /data && chown pwuser:pwuser /data
VOLUME /data
# 不用 root 執行
USER pwuser

EXPOSE 5004
HEALTHCHECK --interval=30s --timeout=5s --start-period=20s --retries=3 \
  CMD python -c "import json,urllib.request;r=urllib.request.urlopen('http://127.0.0.1:5004/health',

CMD ["python", "run.py", "--host", "0.0.0.0", "--port", "5004"]
```

先複製 `requirements-lock.txt` 安裝套件，再複製程式碼：只改程式碼時，套件那層會沿用快取，重新建置只要幾秒。

compose.yml（重點）

複製

```
services:
  app:
    image: devlab/${APP_NAME}:${APP_VERSION}
    pull_policy: never              # 本機建置，不從網路拉
    restart: unless-stopped
    environment:
      ALLOWED_HOSTS: ${APP_NAME}.${LAB_DOMAIN}   # 只接受自己的網址
    volumes:
      - data:/data                  # SQLite 放 volume
    shm_size: 512m                  # Chromium 需要較大的 /dev/shm
    labels:                         # 交給 Traefik 依網址轉發
      - traefik.enable=true
      - traefik.http.routers.${APP_NAME}.rule=Host(`${APP_NAME}.${LAB_DOMAIN}`)
      - traefik.http.services.${APP_NAME}.loadbalancer.server.port=5004
    networks: [devlab]
    deploy:
      resources:
        limits: { cpus: "2", memory: 2g }
    # 注意：沒有 ports:，不直接開放連接埠

volumes:
  data:
    name: chs-warroom_data
    external: true                  # 手動建立；docker compose down -v 不會誤刪資料
```

4 建置：遇到兩個問題

問題 1：建立 Python 虛擬環境失敗

```
#8 1.202 The virtual environment was not created successfully because ensurepip is not
#8 1.202 available.  On Debian/Ubuntu systems, you need to install the python3-venv
#8 1.202 package using the following command.
```

```
#8 1.202     apt install python3.12-venv
#8 ERROR: process "/bin/sh -c python3 -m venv $VIRTUAL_ENV ..." did not complete successfully:
```

解法

錯誤訊息已經寫出答案：Playwright 映像檔沒有附 `venv` 模組。在 `apt-get install` 加上 `python3-venv`。

問題 2：`docker compose build` 失敗，沒有說明原因

```
> docker compose build
failed to execute bake: exit status 1

> docker buildx bake -f compose.yml     # 直接執行底層指令，才看到真正原因
ERROR: additional privileges requested
```

複製

解法

compose 檔在 `D:\lab\apps\chs-warroom`，原始碼在 `D:\StockWarRoomV2`。Buildx 的安全機制不允許讀取 compose 資料夾以外的檔案。所以把**建置和執行分開**：`build.ps1` 直接用 `docker build` 建置，compose 只負責執行建好的映像檔。

```
docker build -f D:\lab\apps\chs-warroom\Dockerfile `
  --build-context certs=D:\lab\apps\chs-warroom\certs `
  -t devlab/chs-warroom:2.0.0 D:\StockWarRoomV2
```

複製

建置結果：映像檔 3.85 GB（大部分是 Chromium 和字型）。

經驗：工具只回「exit status 1」時，改用底層指令或 `--progress plain` 重跑，通常就能看到真正的錯誤訊息。

5 用資料庫副本測試

原生版還在服務同事，測試不能影響它。做法：

1. 用 SQLite 的**線上備份 API** 複製資料庫，程式不用停，副本內容也一致。不要直接複製 `.db` 檔，WAL 模式下可能少掉還沒寫回的資料。
2. 把**副本裡的排程全部停用**，容器版測試期間不會自己去抓資料。
3. 放進 Docker volume，啟動容器。

```
tables: assets=52, fetch_status=174, holidays=27, jobs=63, legacy_weekly=175, logs=723, ...
active schedules before: 9
active schedules after: 0
integrity: ok
```

複製

測試結果

項目	做法	結果
健康檢查	`/health`	✓ healthy
首頁、靜態檔	透過新網址連線	✓ 200
週報資料	同一週的 `/api/report`，容器版和原生版逐字比對	✓ **完全相同**（143 週）
圖片、CSV 匯出	`/api/image` 、 `/api/export`	✓ 200
Host 檢查	用錯誤的 Host 直接打容器	✓ 403 拒絕
證交所連線	容器內呼叫 API	✓ 200
中文字型	Chromium 截一張中文測試頁	✓ 見下圖
MoneyDJ 連線	容器內連線	✗ **憑證錯誤** → 第 6 步



容器內 Chromium 的中文字型測試：字形正常，沒有方框。

6 憑證問題：公司的 HTTPS 檢查

症狀：容器連不上 MoneyDJ，但 Windows 連得上

```
# Python
SSLCertVerificationError: certificate verify failed: self-signed certificate in certificate cha
# Chromium（Playwright）
Page.goto: net::ERR_CERT_AUTHORITY_INVALID at https://concords.moneydj.com/...
# Windows 主機
host moneydj=200
```

用 `openssl` 看容器收到的憑證鏈，發現 MoneyDJ 的憑證**不是 MoneyDJ 自己的 CA 簽的**，而是公司的：

```
0 s:CN = *.moneydj.com
  i:O = CHS, OU = IT, CN = trust.chs.com.tw          ← 公司防火牆重新簽章
1 s:CN = trust.chs.com.tw
  i:DC = tw, DC = com, DC = chsteel-ad, CN = CHS-RootCA
2 s:CN = CHS-RootCA    （自我簽署的公司根憑證）
```

```
# 對照：證交所沒有被攔截，是正常的 TWCA 憑證
 0 s:CN = www.twse.com.tw
   i:CN = TWCA SSL Certification Authority     Verification: OK
```

公司防火牆會攔截部分網站的 HTTPS，換上公司 CA 簽的憑證。加入公司網域的 Windows 信任這張根憑證，容器裡沒有，所以拒絕連線。

解法：把公司根憑證加進映像檔

1. 從 Windows 憑證存放區（`LocalMachine\Root`）匯出 CHS-RootCA。
2. **先核對指紋**，確認匯出的跟容器看到的是同一張，避免加錯憑證：

```
容器看到的：sha256 Fingerprint=5F:6B:BF:E0:09:29:EB:23:...:AC:BA:A4:AE
匯出的檔案：sha256 Fingerprint=5F:6B:BF:E0:09:29:EB:23:...:AC:BA:A4:AE   ✓ 相同
```
複製

3. 加進**三個**信任清單，因為每個元件各管各的：

元件	信任清單	Dockerfile 裡的做法
系統（openssl、urllib）	`/etc/ssl/certs`	`update-ca-certificates`
Python `requests`	自帶的 certifi，不看系統清單	`REQUESTS_CA_BUNDLE` 指向系統清單
Chromium	自己的 NSS 資料庫	`certutil -A` 加到 `~/.pki/nssdb`

重新建置後，Chromium 在容器內成功擷取 MoneyDJ 的兩張圖資，寫進資料庫：

```
capture 20261009 = [{'kind': 'l4', 'bytes': 150961}, {'kind': 'r2', 'bytes': 63153}]
```
複製

測本週時另外出現「MoneyDJ 顯示的日期區間與要求不符」。先別急著修：這週的結束日 10/16 還沒到，MoneyDJ 會自動把日期改成今天，程式的保護機制因此拒絕存檔。正式排程設在週五 19:30，那時結束日就是當天，不會有這個問題。改測上一週就成功了。

經驗：遇到錯誤時先確認原本的版本在同樣條件下會不會也這樣，不要把原本就有的行為誤當成容器化造成的問題。

7 切換上線

選在平日排程（15:30 起）之前的 11:04 切換，並寫成腳本 `cutover.ps1`，讓每一步都可以重複執行，也可以退回：

1. 停止並停用工作排程器的 `\StockWarRoomV2`，確認 5004 埠已經沒有程式在聽
2. 用線上備份 API 做**正式資料庫**的快照，並做完整性檢查
3. 把快照放進 volume，取代測試副本（排程維持原本的啟用設定）
4. 啟動容器，確認 healthy、排程啟用數量與快照一致
5. 停用舊網址 5004 的防火牆規則

退回只要一行：`cutover.ps1 -Rollback`，會把容器期間的資料匯出，再重新啟用原生版。

第一次執行在第 3 步中斷

```
== 1. 停止並停用原生版排程工作 ==
原生版已停止，排程工作已停用
== 2. 資料庫快照 ==
快照：...\backup\native-20261014-110412.db  完整性=ok  週數=142  啟用排程=9
== 3. 匯入 Docker volume ==
ERROR:  Container chs-warroom-app-1 Stopping
```

複製

原因與解法

`Container ... Stopping` 只是正常的進度訊息，但 docker 把它寫到錯誤輸出（stderr），PowerShell 5.1 在嚴格錯誤模式（`$ErrorActionPreference = 'Stop'`）下就當成錯誤中止了。改成 docker 指令一律在寬鬆模式下執行，再檢查結束代碼 `$LASTEXITCODE`。因為腳本可以重複執行，修正後直接重跑。中斷期間服務停了不到 1 分鐘，資料沒有任何影響。

```
== 1. 停止並停用原生版排程工作 ==
目前狀態：Disabled
== 2. 資料庫快照 ==
快照：...\backup\native-20261014-110453.db  完整性=ok  週數=142  啟用排程=9
```

複製

```
== 3. 匯入 Docker volume ==
== 4. 啟動容器並驗證 ==
== 5. 停用舊網址 5004 的防火牆規則（改用新網址） ==
防火牆規則 StockWarRoomV2 Enabled=False
切換完成：http://chs-warroom.10.125.2.14.sslip.io  排程執行中，啟用 9 項
```

切換後的獨立驗證

腳本說成功還不夠，另外逐項比對容器和原本的資料庫：

項目	原生版資料庫	容器版
週報摘要	142	142 ✓
原始資料	1,355	1,355 ✓
執行紀錄	724	724 ✓
10/9 週圖資時間	2026-10-09 22:00:40	相同 ✓（測試時擷取的圖已被正式資料取代）
原生程式	0 個程序，5004 埠無人監聽 ✓	
排程	9/9 啟用，下次執行：今天 15:30 股價行情 ✓	

8 上線後



新網址的戰情室首頁（10/5 ~ 10/9 這一週）。



L4. 券商分點主力進出狀況

MoneyDJ · 指定週區間

中鴻(2014) 券商分點-進出明細

單位：張　最後更新日：2026/10/08

請選擇　自設區間：從 115 年 10 月 5 日 ～ 115 年 10 月 9 日 查詢

買超					賣超				
買超券商	買進	賣出	買超	佔成交比重	賣超券商	買進	賣出	賣超	佔成交比重
國泰-敦南	437	141	296	5.73%	凱基-士林	1	350	349	6.76%
新加坡商瑞銀	294	115	179	3.46%	凱基-台北	83	226	143	2.77%
新光-高雄	173	2	171	3.31%	玉山-大里	2	120	118	2.28%
美商高盛	352	193	159	3.08%	富邦-嘉義	0	106	106	2.05%
新光	185	34	151	2.92%	元大-竹北	0	103	103	1.99%
富邦-岡山	110	0	110	2.13%	華南永昌-岡山	21	83	62	1.2%
凱基-高美館	83	8	75	1.45%	摩根大通	160	218	58	1.12%
凱基-高雄	75	1	74	1.43%	台灣摩根士丹利	181	239	58	1.12%
港商野村	68	0	68	1.32%	富銀-民權	5	62	57	1.1%
元大-永和	61	0	61	1.18%	統一-台中	1	54	53	1.03%
元大-土城永寧	50	0	50	0.97%	元大-高華	28	80	52	1.01%
富邦-新竹	51	4	47	0.91%	富邦-公益	6	57	51	0.99%
台新	56	9	47	0.91%	富邦-高雄	1	51	50	0.97%
元大-鳳中	44	0	44	0.85%	兆豐-鹿港	0	49	49	0.95%
群益金鼎-嘉義	40	0	40	0.77%	富邦-新店	1	47	46	0.89%
合計買超張數			1,572		合計賣超張數				1,354
平均買超成本			16.95		平均賣超成本				16.94

[註1] 合計買超或賣超，為上述家數合計。
[註2] 平均買超或賣超成本，為上述家數合計買賣超金額/上述家數合計買賣超張數。

券商分點要求區間 20261005–20261009 · 擷取／匯入時間：2026-10-09T22:00:40+08:00

R2. 籌碼分佈

MoneyDJ · 籌碼快照

中鴻(2014)籌碼流向

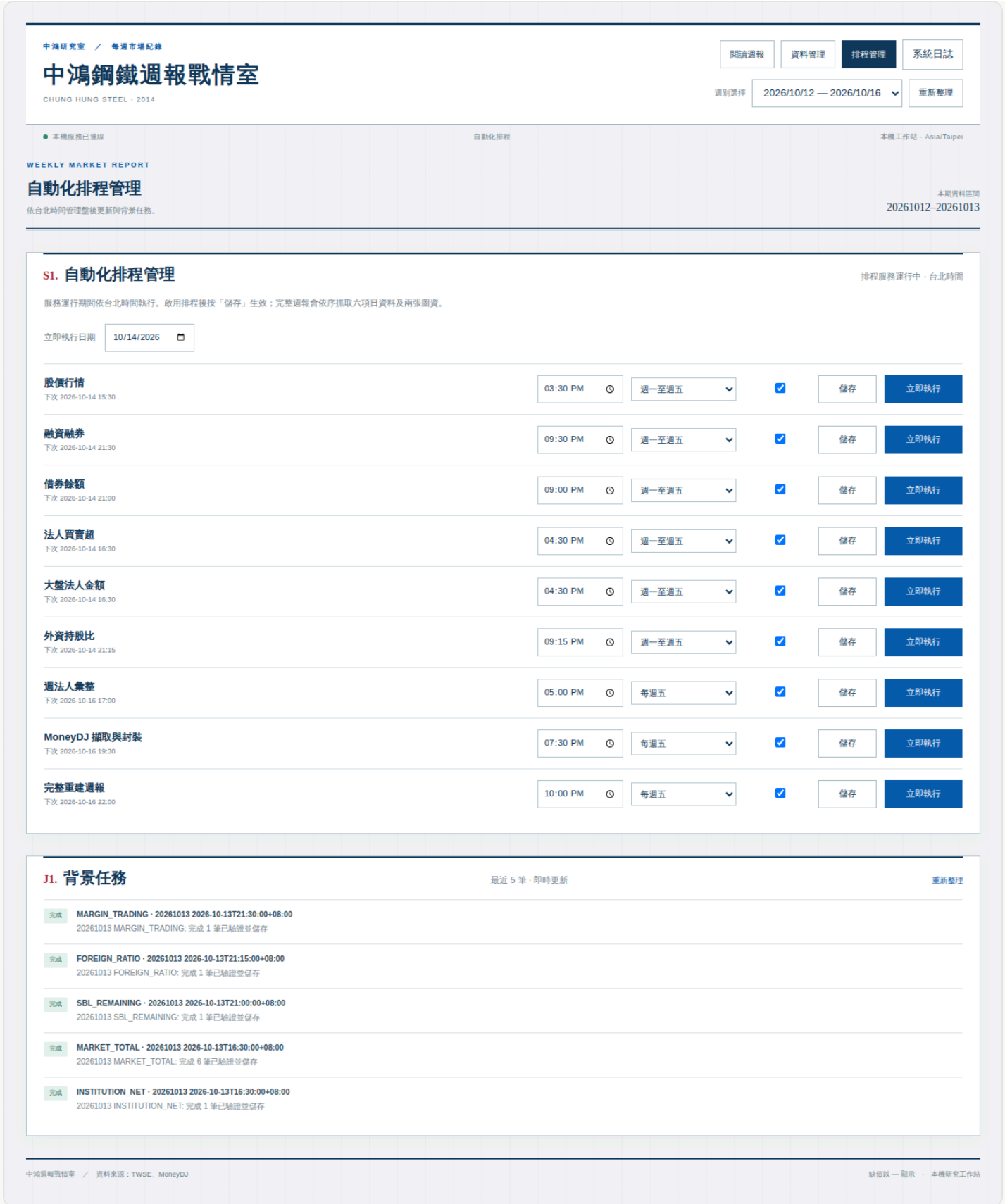
日期：10/08

	最新張數	當日(10/08)增減	佔股本比例
董監持股	582,673		40.59%
外資持股	105,214	378	7.32%
投信持股	4	N/A	0.00%
自營商持股	64,412	63	4.49%
融資餘額	521	-1	0.04%
融券餘額	0	0	

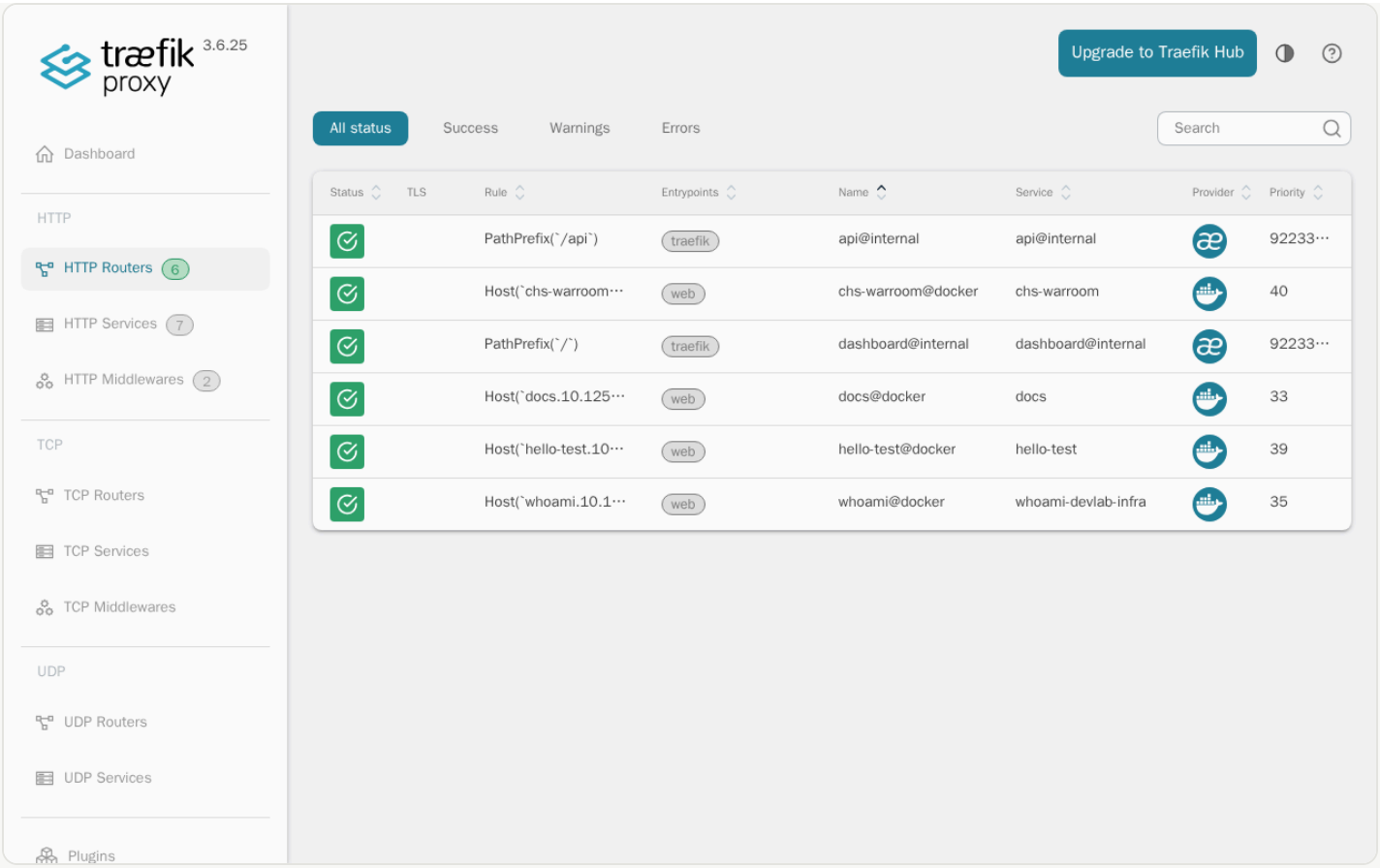
說明：由於董監可能是外資，故股本比例總和有可能超過100%

擷取當下的籌碼流向快照；不是指定歷史週區間 · 擷取／匯入時間：2026-10-09T22:00:41+08:00

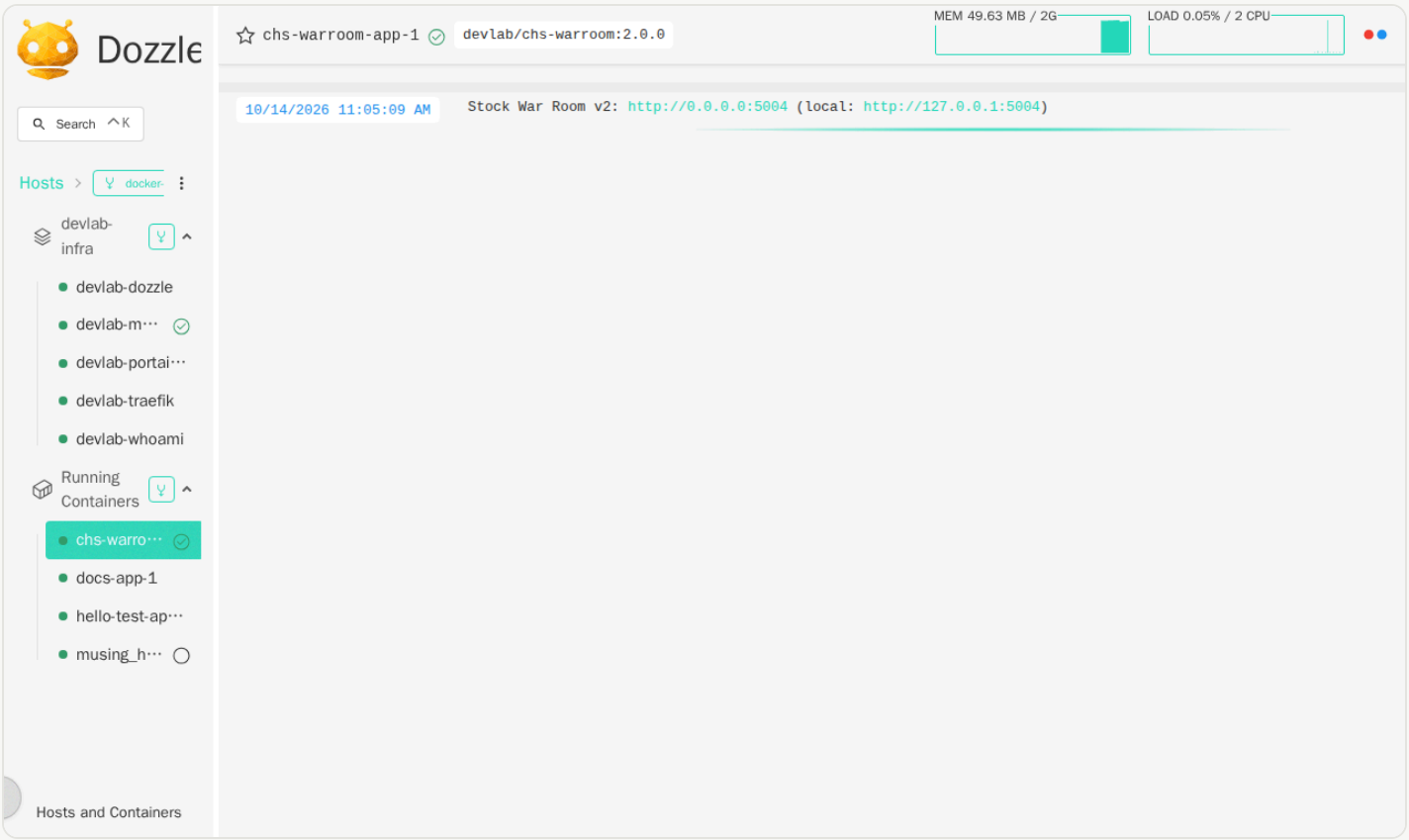
MoneyDJ 圖資：L4 券商分點、R2 籌碼分佈。這是第 6 步解決憑證問題後，在容器裡用 Chromium 擷取的。



排程管理頁：9 項排程都是啟用狀態，下次執行時間正確（台北時間）。



Traefik 儀表板（管理員專用）：`chs-warroom@docker` 依 compose.yml 的 labels 自動建立路由，不用手動設定反向代理。



Dozzle（管理員專用）：即時 log，以及記憶體（約 50 MB / 上限 2 GB）和 CPU 用量。

日常維運

要做什麼	方式
程式改版	`build.ps1` 重新建置 → `docker compose up -d`，避開 15:30～22:00
備份	每天 23:30 自動備份到 `D:\lab\backups\chs-warroom`，保留 30 天
退回原生版	`cutover.ps1 -Rollback`

給你的檢查清單

把自己的程式容器化時，可以照這個實例逐項確認：

1. **找出程式怎麼啟動的。**工作排程器、Windows 服務、啟動資料夾都要查，切換時要一起停用，避免兩份同時跑。
2. **找出寫死的路徑。**例如 `C:\Users\...`、`D:\...`，改成從環境變數讀取。
3. **原始碼和 Docker 設定分開放**，不影響原本的開發流程。
4. **映像檔版本要固定**，並跟程式的套件版本一致（例如 Playwright 1.59.0）。
5. **資料放 volume**；SQLite 尤其不要放 Windows 資料夾掛載。
6. **用資料副本測試**，並停用副本裡會對外動作的功能（排程、寄信）。
7. **跟原本的版本比對輸出**，不要只看「網頁打得開」。
8. **連外部 HTTPS 出現憑證錯誤時**，先查是不是公司的 HTTPS 檢查，再把根憑證加進每個元件的信任清單。
9. **切換寫成可重複執行、可退回的腳本**，選在不影響使用者的時間。
10. **切換後獨立驗證**，並設定好備份。

需要協助請找管理員，見使用說明的聯絡人。
